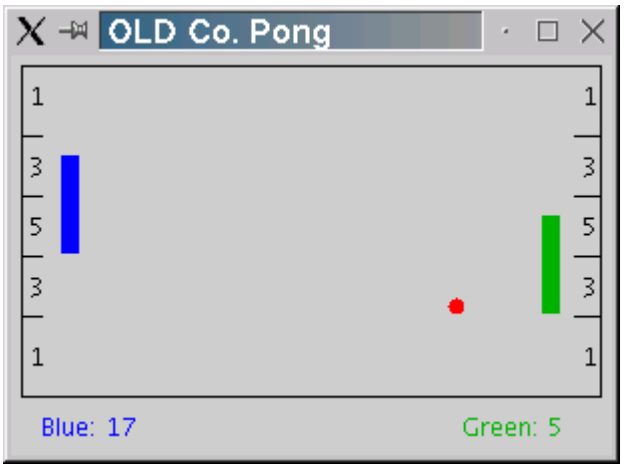
Object-Oriented Language Development Company

The OLD Co Pong Game: Executive Summary

The Object-Oriented Language Development Company (OLD Co.) has begun development of a radically new computer game, with the wholly original name of Pong.



Pong is a two player game in which each player attempts to prevent the pong-ball from reaching the wall behind his pong-paddle. A player deflects the pong-ball before it reaches the wall by placing his pong-paddle into path of the pong-ball. This causes the ball to bounce off of the paddle in the opposite direction. If the ball reaches the wall behind a player's paddle, then the opponent scores. The number of points scored is dependent on the region of the wall contacted by the ball. The closer to the center of the wall the greater the number of points scored.

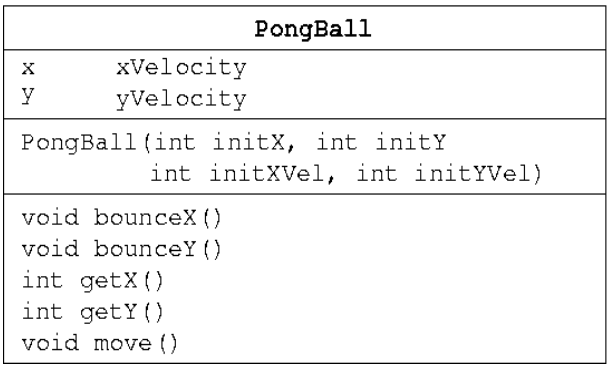
The high-paid software engineers at OLD Co. have spent years designing the Pong program. They have identified the classes that describe the objects the program will use. They have carefully defined the public interfaces of each of the objects. The specifications for these classes were handed off to the programmers who completed the majority of the implementation. However, just before the project was completed, all of the programmers were offered huge salaries by Google and they simply quit on the spot!

I've agreed to write these classes for OLD Co. But being, ever the educator, I decided that writing these classes would be a good experience for you. Of course, I get to keep the check from OLD Co!) The remainder of this document describes the classes you'll need to write to complete the Pong game.

The PongBall Class:

Description:

The `PongBall` class describes the object which the Pong application uses to represent the ball that bounces around the screen. The following class diagram illustrates the information and operations that are defined by the `PongBall` class.



When the Pong application is executed it creates a new `PongBall` object that is positioned at the center of the playing field. Approximately every 10th of a second the Pong application calls the `move()` method on its `PongBall` object causing the ball to update its position. Following the call to `move()` the Pong application uses the `getX()` and `getY()` methods to find the new location of the ball. If the Pong application determines that the ball would have collided with a wall or a paddle it will invoke the `bounceX()` or the `bounceY()` method to change the horizontal or vertical direction of the ball. Finally, the Pong application will draw the ball on the screen at its new location and the process will be repeated again in another 10th of a second.

Implementation:

To implement the `PongBall` class you will need to define instance data for the x and y position of the ball as well as the velocity of the ball in the x and y directions. Your implementation of the `PongBall` class must meet these specifications, so please study them carefully.

`Constructor`: construct a new `PongBall` with the specified initial position and velocity.

Construction Parameters:

- `initX` - the initial X position of the new `PongBall`.
- `initY` - the initial Y position of the new `PongBall`.
- `initXVel` - the initial velocity of the new `PongBall` in the X direction.
- `initYVel` - the initial velocity of the new `PongBall` in the Y Direction.

Public Interface Specification:

- **`void bounceX()`** - Reverse the X direction of this `PongBall`. This method is invoked by the Pong application each time this `PongBall` collides with a vertical obstruction such as a wall or paddle. The X direction can be reversed by changing the sign of the X velocity.

- **`void bounceY()`** - Reverse the Y direction of this `PongBall`. This method is invoked by the Pong application each time this `PongBall` collides with a horizontal

obstruction such as a wall or the top/bottom edge of paddle. The Y direction can be reversed by changing the sign of the Y velocity.

- **`int getX()`** - Return the current X position of this `PongBall`.
- **`int getY()`** - Return the current Y position of this `PongBall`.
- **`void move()`** - Move this `PongBall` according to its current velocity. This method is invoked at a regular rate by the Pong application in order to update the position of the ball. The X position changes by the velocity in the X direction and the y position changes by the velocity in the Y direction.

Testing:

Before we accept your code, we must be certain that it meets the specification. Since you do not know precisely how the Pong application uses the `PongBall` class, it would be very difficult to determine simply by watching the Pong application if the `PongBall` class behaves correctly in every situation. Thus, before adding the `PongBall` class to the Pong application you will need to test its behavior independently by passing a **unit test**.

To test the `PongBall` class you will need to write a class named `PongBallTest` that contains a `main` method. The `main` method in the `PongBallTest` class should create several `PongBall` objects and test their instance methods by comparing the expected state of the object with the actual state.

For example, the following snippet of code will create a `PongBall` object and test the `getX()`, `getY()` and `move()` methods:

```
public class PongBallTest
{
    public static void main(String[] args)
    {

        PongBall ball = new PongBall(10,20,2,4);

        System.out.println("x should be 10, x is: " + ball.getX());
        System.out.println("y should be 20, y is: " + ball.getY());

       ball.move();
       System.out.println("x should be 12, x is: " + ball.getX());
       System.out.println("y should be 24, y is: " + ball.getY());

       ball.move();
       System.out.println("x should be 14, x is: " + ball.getX());
       System.out.println("y should be 28, y is: " + ball.getY());
    }
}
```

Notice that the output of the test program indicates both the correct values and the actual values that are computed. Writing your test programs in this way makes it easy to tell if the class you are testing is working correctly.

You will need to add additional code to the `PongBallTest` class to test the `bounceX()` and `bounceY()` methods.

Integration:

Once you have fully tested your `PongBall` class you can integrate it into the Pong application. To integrate your `PongBall` into the Pong application copy `PongBall.class` from your working directory into the Pong directory, overwriting the old, non-functional `PongBall.class`. (If your program becomes non-functional, you can copy a fresh version of the Pong game from the Q: drive.)

Now when you open the Pong project in BlueJ and start the problem, press the "B" key on the keyboard and the ball should begin bouncing around the playing field.

The PongScore Class:

Description:

The `PongScore` class describes the object that the Pong application uses to keep track of the score for each player. The following class diagram illustrates the information and operations that are defined by the `PongScore` class.

PongScore
score
PongScore()
int getScore() void scorePoints(int points)

When the Pong application is executed it creates two `PongScore` objects, one for the blue player and one for the green player. Each time the ball contacts the end wall behind the blue paddle, the `scorePoints(...)` method of the green player's `PongScore` object is invoked, and vice versa. When invoking the `scorePoints(...)` method, the Pong application passes it an argument (or parameter) corresponding to the number of points indicated in the region of the wall that was contacted. The Pong application will then invoke the `getScore()` method to determine the score to be displayed beside the player's name below the playing field.

Implementation:

Implement the `PongScore` class in the file `PongScore.java` in your working directory. Your implementation must meet the specifications given below.

`Constructor`: construct a new `PongScore` with an initial score of zero points. There are no construction parameters.

Public Interface Specification:

- **`int getScore()`** - Return the current score of this `PongScore` object.

- **`void scorePoints(int points)`** - Increase the current score of this `PongScore` object by points.
  - **`points`** - the number of points to add to the score.

Testing:

To test the `PongScore` class you'll use the Unit Testing framework called JUnit, which is built in to BlueJ. Create a test class named `PongScoreTest`. In your test fixture, you'll need to create at least one `PongScore` object. Your test methods should examine both the initial state of the object after construction, as well as calling the `scorePoints(...)` method with several different arguments. Use assertions to compare the values that you expect your `PongScore` object to contain with its actual values. Correct your implementation if the two values are different.

Integration:

Once you have fully tested your `PongScore` class you can integrate it into the Pong application by copying the `PongScore.class` file from your working directory into the Pong project, overwriting the old, non-functional `PongScore.class`. When you open the Pong project in BlueJ, run the Pong application and press the "B" key, the ball will again begin to bounce around the playing field. However, now each time the ball contacts the end wall the opponent's score will be incremented by the appropriate number of points.

The PongPaddle Class:

Description:

The `PongPaddle` class describes the objects that the Pong application uses to keep track of the size and location of the pong paddles. The following class diagram illustrates the information and operations that are defined by the `PongPaddle` class. Each `PongPaddle` has an `(x,y)` position that indicates the position of the top-left corner of the paddle. Each `PongPaddle` also has a **`width`** and a **`height`**.

PongPaddle
x y width height
PongPaddle(int top, int left, int w, int h)
int getBottomY() int getLeftX() int getRightX() int getTopY() void moveDown(int d) void moveUp(int d)

When the Pong application is run it creates two new `PongPaddle` objects: one for the blue player positioned at the left end of the

playing field, the other for the green player positioned at the right end of the playing field. When the blue player presses the "A" key, the Pong application invokes the `moveUp(...)` method of the blue player's `PongPaddle` object. Conversely, when the blue player presses the "Z" key the Pong application invokes the `moveDown(...)` method. Similar actions are taken using the green player's `PongPaddle` object when the "M" and "K" keys are pressed. The Pong application then uses the `getLeftX()`, `getTopY()`, `getRightX()` and `getBottomY()` methods to determine where on the screen the blue and green paddles should be drawn. These methods are also used by the Pong application to determine when the ball collides with one of the paddles.

Implementation:

Implement the `PongPaddle` class in the file `PongPaddle.java` in your lab5 directory. Your implementation must meet the specifications given below.

`Constructor`: Construct a new `PongPaddle` at the specified position with the specified width and height.

Construction Parameters:

- `top - the Y coordinate of the top left corner of the new PongPaddle.`
- `left - the X coordinate of the top left corner of the new PongPaddle.`
- `w – the width of the new PongPaddle, measured in pixels.`
- `h – the height of the new PongPaddle, measured in pixels.`

Public Interface Specification:

- **`int getBottomY()`** - Return the Y coordinate of the bottom edge of this `PongPaddle`.

- **`int getLeftX()`** - Return the X coordinate of the left edge of this `PongPaddle`.

- **`int getRightX()`** - Return the X coordinate of the right edge of this `PongPaddle`.

- **`int getTopY()`** - Return the Y coordinate of the top edge of this `PongPaddle`.

- **`void moveDown(int pixels)`** - Move the position of this `PongPaddle` down by the specified number of pixels..
  - **`pixels`** - the number of pixels to move down.

- **`void moveUp(int pixels)`** - Move the position of this `PongPaddle` up by the specified number of pixels..
  - **`pixels`** - the number of pixels to move up.

Testing:

To test the `PongPaddle` class write a class named `PongPaddleTest` using BlueJ's JUnit testing facility. Be sure to create several `PongPaddle` objects and to call all of the instance methods several times with several different arguments (when appropriate).

Integration:

Once you have tested your `PongPaddle` class you can integrate it into the Pong application by copying the `PongPaddle.class` file from your working directory into your Pong directory.

When the Pong application is run from BlueJ, pressing the "B" key begins the ball bouncing as before. However, now when the "A" or "Z" keys are pressed the blue paddle will move up or down. Similarly, when the "K" or "M" keys are pressed the green paddle will move up or down.

Congratulations! You now have a complete working Pong application!